
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We introduce FLOWTIME, a generative framework for probabilistic forecasting in multivariate timeseries settings. From historical measurements and optional future covariates, the forecasting problem is treated as drawing samples from a learned conditional law over future trajectories. Rather than modeling the full horizon as one monolithic object, we express the joint distribution of future observations as a sequence of conditional densities, where a shared flow maps a simple base distribution to the next-step observation distribution under the available covariate context. We instantiate this construction with flow matching (FM), which supports scalable, simulation-free learning of the required transformations. This combination of autoregressive factorization and the FM objective preserves practical strengths associated with sequential forecasting models—including extrapolation over horizons beyond the training target, compact parameterization, and calibrated uncertainty estimates—while retaining the ability of transport-based generative models to represent complex, multi-modal conditional distributions. Across stochastic dynamical systems and real-world forecasting benchmarks, FLOWTIME provides evidence that this formulation is broadly useful for probabilistic timeseries prediction without changing the underlying experimental tasks or assumptions.

1 INTRODUCTION

A central problem across modern machine learning applications is probabilistic timeseries forecasting: estimating how an observed process may evolve while quantifying uncertainty over future outcomes. This formulation appears in many scientific, industrial, and societal domains where decisions depend not only on point predictions but also on plausible future trajectories (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

Deep generative models provide a flexible way to address this objective by representing forecasts as samples from a future probability density conditioned on past observations (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021). Within this broader class, diffusion and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have achieved strong results on several forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kollovich et al., 2023; Meijer & Chen, 2024). Their success demonstrates the value of expressive conditional generative models for forecasting, but their iterative stochastic denoising procedures often impose substantial computational costs in both training and inference.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) offers a general transport-based alternative for conditional generative modeling, with diffusion models appearing as one important special case. Instead of relying on iterative stochastic denoising along a discretized reverse process, FM learns deterministic probability paths that transform arbitrary base distributions into target distributions through continuous normalizing flows. With suitable probability-path design, FM avoids handcrafted noise schedules and long ancestral sampling chains, making it attractive for forecasting settings where both sample quality and computational efficiency matter.

Recent work has begun to apply FM to timeseries modeling and probabilistic forecasting (Tamir et al., 2024; Hu et al., 2024; Kollovich et al., 2024), reporting competitive empirical performance and improved efficiency relative to diffusion-based approaches. Many of these formulations learn the conditional distribution of an entire fixed-horizon future window given a fixed-horizon context

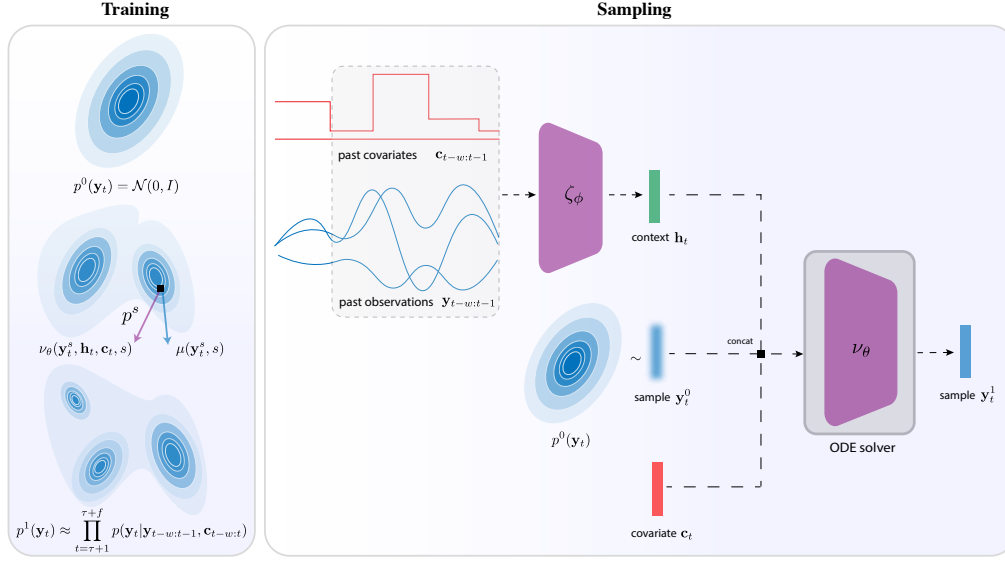


Figure 1: Overview of the training and sampling workflow used by the proposed autoregressive flow-matching forecaster. During training, a probability path $(p^s)^{0 \leq s \leq 1}$ connects a base distribution p^0 to the target distribution p^1 . The path is generated by a velocity field μ , and the model learns it by regressing μ with a neural network ν_θ that receives a path sample $\mathbf{y}_t^s$ at flow step s , a context vector $\mathbf{h}_t$ summarizing past observations and covariates, and the current covariate $\mathbf{c}_t$. At sampling time, a draw from p^0 is transported to the target distribution by integrating the learned velocity field with an ODE solver until $s = 1$.

window. This design can make training and sampling efficient for the prescribed horizon, but it can also narrow the model’s operational scope: extrapolating beyond the training window becomes less natural, uncertainty estimates may become miscalibrated, and the optimization problem must represent intricate time-dependent probability paths over the whole future block. One response is to introduce informed Gaussian-process priors to simplify and accelerate optimization (Kollovich et al., 2024), but that particular strategy limits the setting to univariate forecasting.

We propose FLOWTIME as a simple FM-based alternative whose scope includes high-dimensional multivariate forecasting without requiring informed priors. Rather than treating the future horizon as a single fixed block, FLOWTIME decomposes forecasting into a sequence of conditional distributions, one for each future time step given recent observations and covariates. A shared flow then transports a simple base distribution to the next-step observation distribution. This autoregressive construction keeps the method compatible with variable forecasting horizons, supports extrapolation beyond the window used for training, and yields uncertainty estimates through sequential conditional sampling while reducing the complexity of each learned transport problem.

We evaluate this formulation on both simulated stochastic dynamical systems and real-world time-series benchmarks, showing that the same modeling principle performs well across controlled scientific dynamics and applied forecasting datasets.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a generative-modeling framework that makes continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) practical at large scale. CNFs view data transformations as solutions of ordinary differential equations (ODEs), yielding invertible mappings whose likelihoods remain tractable.

As in CNFs, FM seeks a time-dependent diffeomorphic map on the data space Ω , referred to as a flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. The flow transports a source sample $\mathcal{X}^0 \sim p$ to a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ so that $\mathcal{X}^1 \sim q$ for a desired target distribution q . This transport is defined by the following initial

value problem:

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ is the vector field that specifies the flow velocity and induces a probability path $(p^s)_{0 \leq s \leq 1}$, with each p^s a distribution over Ω and $p^0 = p$. Learning a valid vector field μ therefore amounts to constructing a path whose terminal distribution satisfies $p^1 = q$.

Whereas standard CNFs learn the vector field through likelihood maximization and therefore require solving and differentiating through Eq. (1) during training, FM learns a parameterized vector field ν_θ by regression against vector fields associated with fixed conditional probability paths. This perspective yields the tractable conditional flow matching (CFM) objective:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0,1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

After minimizing this objective, samples from q are obtained by solving Eq. (1) with the trained ν_θ . Because training avoids backpropagating through the ODE solver, FM scales to high-dimensional generative tasks such as image and video modeling (Esser et al., 2024; Polyak et al., 2024). Different choices of probability paths and conditioning mechanisms then instantiate a broad family of transport-based generative models within the same framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ be an n -dimensional observation at time τ . Given l previous measurements including the current observation, $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, we aim to generate probabilistic forecasts for the next f values, $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$. The formulation also allows observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ with $\mathbf{c}_\tau \in \mathbb{R}^c$, covering settings where future exogenous information is available. Formally, given $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, the objective is to sample from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$.

We factorize this conditional law autoregressively over future time steps under an order- w Markov assumption:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ is the history size. This reduces the modeling target to the conditional distributions appearing in Eq. (3). Rather than estimating them by maximum likelihood, as is common in autoregressive forecasting, we learn these conditionals with the flow matching framework (Lipman et al., 2022).

The factorization is useful beyond a single fixed-horizon benchmark because it separates a difficult joint forecasting problem into reusable next-step conditional models $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$. Each conditional can be trained with teacher forcing, so the training computations remain parallel across time points even though sampling is sequential. The window size w controls the assumed temporal memory and therefore mediates a practical trade-off between model capacity and computational cost. Finally, uncertainty is propagated through an explicit chain of conditional distributions, making the resulting forecast distribution interpretable over extended horizons.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For each future time step $t \in [\tau+1, \tau+f]$, we define a target probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ whose endpoints are $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, where $p_{\mathcal{D}}$ denotes the empirical distribution. The same construction applies across horizon positions because the next-step target is always represented in $\mathbb{R}^n$. A flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ generates this path along s through the ordinary differential equation (ODE):

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the vector field used to construct a flow satisfying the continuity equation.

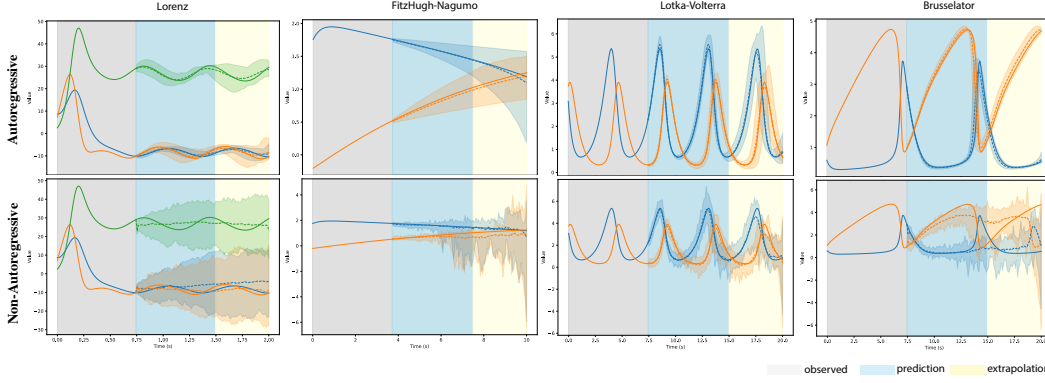


Figure 2: Forecast samples comparing autoregressive flow matching with standard flow matching on four stochastic dynamical systems. Solid lines show ground truth trajectories, and dashed lines show mean predictions with 95% confidence intervals. Each panel includes both the in-window prediction regime and the extrapolation regime beyond the training target.

To obtain an objective that remains tractable for these conditional forecasting distributions, we express the marginal path as a mixture of conditional probability paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t|\mathbf{z})p(\mathbf{z})d\mathbf{z} \quad (5)$$

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning random variable drawn from a data coupling $\pi_{0,1}$. Theorem 2 in (Lipman et al., 2022) establishes that the conditional velocity fields $\mu(\mathbf{y}_t, s|\mathbf{z})$ associated with $p^s(\mathbf{y}_t|\mathbf{z})$ recover the corresponding marginal velocity field $\mu(\mathbf{y}_t, s)$.

Following (Tong et al., 2023), we set $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are samples from the source and target endpoints. The conditional probability path is then

$$p^s(\mathbf{y}_t|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

with marginals satisfying $p^0(\mathbf{y}_t|\mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t|\mathbf{z}) = p^1(\mathbf{y}_t)$ as $\sigma^2 \rightarrow 0$. The associated conditional velocity field simplifies to $\mu(\mathbf{y}_t, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This straight-line path in Euclidean space provides closed-form targets for every next-step conditional distribution, which keeps training and sampling efficient. Once the velocity field is learned, samples from $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ are produced by solving the ODE in Eq. (4) to $s = 1$.

3.2 TRAINING

As in standard FM, we learn the velocity field by regressing it with a neural network ν_θ with parameters θ under the conditional flow matching objective in Eq. 2. To adapt this objective to forecasting contexts, we introduce a context vector $\mathbf{h}_t \in \mathbb{R}^h$ that encodes the local history at time t :

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural network with parameters ϕ . The resulting learning objective is

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t|\mathbf{z})} \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

We train this objective by randomly sampling a time point t , its context window, and covariates from $\mathcal{D}$, then jointly optimizing ϕ and θ with stochastic gradient descent. This training procedure is analogous to teacher forcing (Williams & Zipser, 1989): ground-truth histories are used during learning instead of previously generated model outputs. Consequently, all horizon positions contribute training examples in parallel, while sampling can still roll forward autoregressively. Algorithm 1 summarizes this autoregressive flow-matching training loop.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
- 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
- 5: Compute target: $\mu(\mathbf{y}_t^s, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
- 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
- 7: Update θ, ϕ
- 8: **end while**
- 9: **return** ν_θ, ζ_ϕ

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s)$, $\psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0$, $s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

3.3 INFERENCE

After training, the model can generate probabilistic forecasts for new histories $\mathbf{Y}_l$ and covariates $\mathbf{C}$ over any specified horizon of f future timestamps. We use the sampling procedure in Algorithm 2, rolling the context window forward as each new sample is appended. Repeating this procedure yields Monte Carlo samples from the forecast distribution, from which empirical predictive quantiles and other uncertainty summaries can be computed.

4 EXPERIMENTS

We evaluate FLOWTIME in two complementary regimes: controlled stochastic dynamical systems, where extrapolation behavior can be studied explicitly, and real-world forecasting datasets, where the model is compared with established statistical, autoregressive, diffusion, and flow-matching baselines.

4.1 DYNAMICAL SYSTEMS

Data We tested FLOWTIME on five stochastic dynamical-system forecasting problems that span chaotic, excitable, ecological, chemical, and oscillatory dynamics: stochastic versions of the Lorenz system (Lorenz, 1963), FitzHugh–Nagumo model (FitzHugh, 1961), Lotka–Volterra system Lotka (1925), Brusselator (Lefever & Prigogine, 1968), and Van der Pol oscillator (van der Pol, 1926). For each system, trajectories were generated by numerically solving the corresponding stochastic differential equations with randomly sampled initial conditions and Brownian motion realizations. Appendix A.1 gives the full data-generation details.

Implementation details For this family of simulated systems, ζ_ϕ is a two-layer bi-directional LSTM with 64 hidden units and ν_θ is an MLP with three hidden layers of size 64. The flow step s is encoded with 16-dimensional Fourier positional embeddings using a fixed frequency set, and the context length w is set equal to the prediction length. Models were optimized with Adam using learning rate 0.003 and batch size 128; training to convergence took ~ 7 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines To isolate the role of temporal factorization, we compare FLOWTIME with a non-autoregressive flow-matching baseline that directly models the joint future distribution conditioned on past observations and covariates, $p^1 = p(\mathbf{Y}_f | \mathbf{Y}_t, \mathbf{C})$. This baseline is a direct ablation of the proposed autoregressive decomposition and closely matches the modeling style of (Kolloviev et al., 2024). Appendix B provides the corresponding formulation and implementation details.

Evaluation To evaluate both within-horizon prediction and beyond-horizon generalization, each trajectory is partitioned into three segments: (1) an observation window used as historical input, (2) a prediction window available during training as the supervised target, and (3) an extrapolation window that is never included in training targets. We report the continuous rank probability score (CRPS) (Winkler et al., 1996) to assess probabilistic calibration and distributional accuracy, and the normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006) to assess point-forecast accuracy.

Table 1: Test-set comparison between the autoregressive factorization used by FLOWTIME and a non-autoregressive (Non-AR) flow-matching factorization across five stochastic dynamical systems. CRPS and NRMSE are reported for both the prediction window and the extrapolation window, highlighting performance within and beyond the training target horizon. Values are means and standard deviations over five random seeds.

System	Factorization	Prediction		Extrapolation	
		CRPS ↓	NRMSE ↓	CRPS ↓	NRMSE ↓
Lorenz	Non-AR	0.147±0.012	0.242±0.018	0.120±0.009	0.301±0.024
	FLOWTIME	0.090±0.008	0.017±0.003	0.137±0.011	0.048±0.006
FitzHugh-Nagumo	Non-AR	0.165±0.014	0.131±0.011	0.214±0.016	0.306±0.022
	FLOWTIME	0.144±0.013	0.090±0.007	0.146±0.012	0.278±0.019
Lotka-Volterra	Non-AR	0.130±0.011	0.266±0.019	0.136±0.012	0.502±0.033
	FLOWTIME	0.126±0.010	0.189±0.015	0.123±0.009	0.339±0.026
Brusselator	Non-AR	0.217±0.016	0.658±0.042	0.238±0.019	1.046±0.057
	FLOWTIME	0.125±0.011	0.031±0.004	0.109±0.008	0.023±0.003
Van der Pol	Non-AR	0.143±0.012	0.161±0.014	0.211±0.018	0.510±0.031
	FLOWTIME	0.225±0.017	0.051±0.005	0.282±0.021	0.101±0.009

Results Table 1 shows that the autoregressive factorization used by FLOWTIME improves forecasting across a range of stochastic dynamical systems rather than only on a single simulated process. In the prediction window, the AR model obtains lower NRMSE than the non-AR baseline on all five systems, with especially large reductions for Lorenz (0.017 vs 0.242) and Brusselator (0.031 vs 0.658), corresponding to 93% and 95% lower error. The AR factorization also improves CRPS on four of the five systems, with Van der Pol being the exception where the non-AR baseline has lower CRPS. The same pattern largely carries into the extrapolation window: AR maintains the best NRMSE on every system, including a 98% reduction on Brusselator (0.023 vs 1.046). These results support the claim that next-step flow matching with autoregressive conditioning can represent diverse stochastic dynamics while preserving useful generalization behavior beyond the training target horizon.

4.2 REAL-WORLD DATASETS

Data We further evaluated FLOWTIME on real-world forecasting tasks covering both univariate and multivariate timeseries and both hourly and daily sampling frequencies from GluonTS (Alexandrov et al., 2020). The benchmark suite consists of Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019). These datasets broaden the empirical setting from controlled SDE simulations to applied forecasting domains; Appendix A.2 provides additional dataset details.

Implementation details For the real-world benchmarks, ζ_ϕ is a three-layer bi-directional LSTM with 64 hidden units. The velocity network ν_θ is a residual neural network with five residual blocks implemented using 1-D convolutional layers and gated activations (Van Den Oord et al., 2016; Kong et al., 2020); this architecture follows the design used in (Rasul et al., 2021) with fewer residual blocks. Fourier positional embeddings map the flow step s to a 32-dimensional vector using a fixed frequency set, and the same embedding strategy is used for time-dependent covariates c . Models were optimized with Adam using learning rate 0.001 and batch size 128; depending on the dataset, convergence required 40 – 100 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines We compare FLOWTIME with established methods spanning several forecasting model classes. The comparison includes traditional statistical methods, Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008); neural autoregressive models, DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016); diffusion-based probabilistic forecasters, TSDiff (Kollovich et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021); and the flow-matching baseline TSFlow (Kollovich et al., 2024).

Table 2: Test-set CRPS comparison (lower is better) across five real-world forecasting datasets. FLOWTIME values report the mean and standard error over five independent training and evaluation runs, while baseline results are taken from (Kollovich et al., 2024). Best scores are shown in **bold**, and second-best scores are underlined.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060±0.000	0.013±0.000	0.512±0.000	0.221±0.000	0.423±0.000
ARIMA	0.344±0.000	0.013±0.000	0.558±0.003	0.486±0.000	0.654±0.000
ETS	0.056±0.000	0.008±0.000	0.550±0.000	0.492±0.000	0.651±0.000
DeepAR	0.051±0.000	0.013±0.004	0.429±0.015	0.103±0.000	0.215±0.003
WaveNet	0.058±0.008	0.012±0.001	0.360±0.000	0.099±0.000	0.207 ±0.000
CSDI	0.051±0.000	0.013±0.000	0.360±0.000	0.152±0.000	0.318±0.012
SSSD	0.048±0.000	<u>0.010</u> ±0.000	0.354±0.024	0.107±0.002	<u>0.209</u> ±0.000
TSFlow	0.045±0.001	0.009 ±0.001	0.343±0.002	0.083 ±0.000	<u>0.227</u> ±0.000
FLOWTIME	0.042 ±0.001	0.009 ±0.001	0.284 ±0.002	<u>0.089</u> ±0.000	0.243±0.002

Results Table 2 compares FLOWTIME with the baseline suite on five real-world timeseries datasets. FLOWTIME attains the best CRPS on Electricity, Exchange, and Solar, indicating that the same autoregressive flow-matching formulation transfers across multiple applied data domains. On Electricity, it achieves CRPS 0.042, a 6.7% improvement over TSFlow at 0.045. On Exchange, it matches TSFlow at 0.009 while outperforming the remaining statistical and neural baselines. The largest gain appears on Solar, where CRPS 0.284 corresponds to a 17.2% reduction relative to TSFlow at 0.343. FLOWTIME ranks second on Traffic with CRPS 0.089, slightly behind TSFlow at 0.083 but ahead of the other reported baselines, and remains competitive on Wikipedia with CRPS 0.243, trailing WaveNet (0.207) and SSSD (0.209). Overall, these results show that the proposed factorization is not confined to synthetic dynamics: it yields state-of-the-art or competitive probabilistic forecasts across heterogeneous real-world benchmarks.

5 DISCUSSION

We introduced FLOWTIME, a probabilistic forecasting model that combines autoregressive conditioning with flow matching to produce calibrated multivariate timeseries forecasts. The key modeling choice is to replace a fixed future-window transport problem with a sequence of reusable next-step conditional transports, thereby pairing the temporal inductive bias of autoregressive forecasting with the expressiveness of flow-based generative modeling. The empirical results span both simulated stochastic dynamical systems and real-world benchmark datasets, suggesting that the approach is applicable across multiple forecasting regimes under the assumptions studied here. Relative to a standard flow-matching baseline that models entire future windows jointly, the autoregressive decomposition improves prediction accuracy and uncertainty behavior in the dynamical-system experiments, with benefits that persist into extrapolation windows. On real-world datasets, FLOWTIME is state-of-the-art or competitive against statistical, neural autoregressive, diffusion, and flow-matching

baselines, supporting its use as a general probabilistic forecasting architecture within the evaluated conditions.

These conclusions remain bounded by the experimental setting. Because sampling is autoregressive, inference is sequential and can be slower than fully parallel non-autoregressive generation, which may matter in real-time forecasting deployments. The current implementation also performs flow matching directly in data space; for very high-dimensional, irregularly sampled, or partially observed systems, latent-space or observation-model extensions may be more appropriate. Future work could study such variants, broaden the treatment of irregular sampling, and connect the forecast distributions to downstream decision-making under uncertainty. Within the evaluated scope, FLOWTIME illustrates how flow-based generative models can improve both accuracy and uncertainty quantification when paired with an autoregressive temporal factorization.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.

-
- Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.
- Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint arXiv:1605.06432*, 2016.
- Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for probabilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–28364, 2023.
- Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth International Conference on Learning Representations*, 2024.
- Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.
- Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, pp. 95–104, 2018.
- R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.
- Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical Transactions of the Royal Society A*, 379(2194):20200209, 2021.
- Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2022.
- Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.
- Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.
- Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.
- Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv preprint arXiv:2401.03006*, 2024.
- Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media foundation models. *arXiv e-prints*, pp. arXiv–2410, 2024.
- Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Multivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint arXiv:2002.06103*, 2020.
- Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In *International Conference on Machine Learning*, pp. 8857–8868. PMLR, 2021.
- David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus. High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances in Neural Information Processing Systems*, 32, 2019.
- David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3): 1181–1191, 2020.

486 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
487 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*,
488 pp. 2256–2265. PMLR, 2015.

489 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
490 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint*
491 *arXiv:2011.13456*, 2020.

492
493 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
494 ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-
495 ative Modeling*, 2024.

496 Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
497 diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-
498 cessing Systems*, 34:24804–24816, 2021.

499 Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrod Rector-
500 Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
501 with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

502
503 Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves,
504 Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for
505 raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.

506
507 B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical*
508 *Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.

509 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
510 neural networks. *Neural Computation*, 1(2):270–280, 1989.

511
512 Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B
513 Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring
514 rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.

515 Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time
516 series foundation models: Generalizing time series representation with large language model. In
517 *ArXiv:2104.00164*, 2024.

518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539

A DATASETS

A.1 DYNAMICAL SYSTEMS

We generated training and testing data from five classical stochastic dynamical systems—Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator, and Van der Pol—by numerically integrating the corresponding stochastic differential equations (SDEs). This set covers several qualitatively different dynamics while sharing the common SDE form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

where f is the deterministic drift, Θ are the system parameters, σ is a constant diffusion term, and $\mathbf{W}(t)$ is standard Brownian motion. For each system, we sampled 2400 random initial conditions from the uniform range in Table 3 and generated one trajectory per initial condition using the Euler-Heun method with fixed step size. Each trajectory contains 200 equally spaced time points: the first 75 form the observed history, the next 75 define the prediction target, and the final 50 constitute the extrapolation segment. Of the 2400 generated trajectories, 2000 were used for training and 400 for testing.

Table 3: Specification of the stochastic dynamical systems used to evaluate forecasting across several families of continuous-time dynamics. $\mathbf{f}(\mathbf{x})$ denotes the drift function, Θ the system parameters, σ the constant diffusion term, and $p(\mathbf{x}(0))$ the initial condition distribution. Each system was integrated over the interval $[t_0, t_1]$ with 200 steps using the Euler-Heun method.

System	$\mathbf{f}(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh-Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka-Volterra		$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\beta = 0.9$			
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\gamma = 0.8$			
Brusselator		$\delta = 1.8$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$A = 1.0$			
	$f_2 = B x_1 - x_1^2 x_2$	$B = 3.0$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

We used five GluonTS datasets (Alexandrov et al., 2020) to evaluate model performance across applied forecasting settings with different dimensionalities, supports, and temporal resolutions. All experiments use the versions preprocessed as in (Salinas et al., 2019), and Table 4 summarizes their main properties.

Table 4: Properties of the real-world datasets used to evaluate forecasting across different dimensions, domains, sampling frequencies, training lengths, and prediction horizons: dimension n , domain Ω , frequency, total training timesteps, and prediction length.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting We use the same conditional forecasting setup as Section 3: given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, the goal is to model $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. The non-autoregressive baseline differs only in scope of the transported object: instead of decomposing time, it models the full future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ as one joint random variable:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation can preserve correlations among all future time steps directly, but it does so by learning a higher-dimensional conditional distribution.

Training The baseline constructs a trajectory-level probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is block diagonal. This prior can be interpreted as independent Brownian motion processes for each dimension, with Brownian motion defined as a stochastic process $W: [\tau + 1, \tau + f] \rightarrow \mathbb{R}^n$.

The conditional probability path is a linear interpolation bridge with Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$, $\mathbf{Y}_f^0 \sim p^0$, and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field is

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

As $\sigma^2 \rightarrow 0$, this expression reduces to the straight-path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. The learned velocity field ν_θ therefore operates on the entire future trajectory rather than on a single next-step observation. Its context encoder remains analogous to Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ processes the past observations together with the relevant past and future covariates. The training objective is

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

where ν_θ is implemented as a sequential neural network, such as a Transformer or RNN, that processes the full future block. Sampling then requires solving the trajectory-level ODE

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

with a numerical solver. The result is a joint sample from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ generated without autoregressive decomposition.

Implementation Details We used this baseline for the stochastic dynamical-system forecasting experiments to test whether joint future-window transport is sufficient in the same data regime. In this setup, ν_θ is a 4-layer bi-directional LSTM with 128 hidden units, and the remaining training parameters match Section 4.1. Training to convergence took ~ 2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant The non-autoregressive baseline and FLOWTIME define different generalization trade-offs over the same forecasting target. The autoregressive method factorizes the joint distribution through a Markovian decomposition across time steps, whereas the non-autoregressive baseline treats the full future trajectory $\mathbf{Y}_f$ as a single high-dimensional random variable. This allows direct representation of cross-temporal dependencies, but it requires learning a distribution over $\mathbb{R}^{n \times f}$. Their computational profiles also differ: the non-autoregressive model processes complete trajectories through a sequential velocity network, enabling parallel generation of all future time points while increasing memory requirements. Their uncertainty mechanisms differ as well. The autoregressive model exposes uncertainty propagation through the chain of conditional distributions, whereas the non-autoregressive model represents joint uncertainty through trajectory-level sampling without an explicit error-accumulation mechanism. Thus, the two formulations are complementary: the autoregressive variant emphasizes window-based reuse and interpretable propagation under a finite-memory assumption, while the non-autoregressive baseline emphasizes full-horizon joint modeling at higher trajectory-level complexity.

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
- 3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$
- 5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
- 6: Compute target $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$
- 7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
- 8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ

- 1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
- 2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
- 3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
- 4: Return $\psi(\mathbf{Y}_f, 1)$

C EVALUATION METRICS

This section defines the metrics used to evaluate forecast quality across the simulated and real-world settings. Given an n -dimensional future time series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ and either its predictive distribution or samples from the probabilistic model, we report the Continuous Ranked Probability Score (CRPS) and the Normalized Root Mean Squared Error (NRMSE).

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

CRPS is a proper scoring rule for probabilistic forecasts and is applicable whenever the full predictive distribution, or an empirical sample approximation to it, is available. For a univariate random variable X with cumulative distribution function (CDF) F_X and observed value x , CRPS is defined as

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. CRPS evaluates alignment between the entire predictive distribution and the observation x , so lower values indicate better calibrated and more accurate probabilistic forecasts.

For a forecasting horizon, CRPS is computed at each future time step and then averaged:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ is the predicted CDF, or an empirical CDF estimated from samples, at horizon k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

Root Mean Squared Error (RMSE) measures point-forecast accuracy and can be applied to the predictive mean or another point summary of the forecast distribution. For predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, RMSE is

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To compare errors across datasets or variables with different magnitudes, we report the Normalized RMSE (NRMSE). Here the normalization uses the standard deviation of the ground-truth series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations. Other normalizations, such as the data range ($\max(y) - \min(y)$), may be appropriate in other applications; under this convention, lower NRMSE indicates more accurate point forecasts relative to the variability of the series.